

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS  
COURSE CODE: CSA07  
NAME: P. Reddy Suhail  
REG NO:192312671L

---

### COURSE OUTCOME COVERED

**CO5:** Measure network performance using key metrics with simulation tools.  
(BL4,BL5)

*Assessment Tool Weightage: 30%*

---

### Annexure A

## Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

### ALGORITHM Network Performance

INPUT total data = 8000

bits INPUT time = 4

seconds INPUT packets sent

= 200 INPUT packets

received = 180

IF time == 0 THEN

DISPLAY "Error: Time cannot be zero"

STOP

END IF

IF packets sent == 0 THEN

DISPLAY "Error: Total packets sent cannot be  
zero" STOP

END IF

throughput = total data / time

packet loss = ((packets sent - packets received) / packets sent) \* 100

DISPLAY "----- Network Performance Summary- -"

DISPLAY "Total Data Received: ", total data, "  
bits" DISPLAY "Time : ", time, " seconds"

DISPLAY "Packets Sent : ", packets sent

DISPLAY "Packets Received : ",

packets received DISPLAY "Throughput

: ", throughput, "

bps" DISPLAY "Packet Loss : ",

packet loss, "%"

IF throughput > 1500 AND packet loss < 10

THEN DISPLAY "Network Classification:

Efficient" ELSE

DISPLAY "Network Classification: Inefficient"

END IF

DISPLAY " "

END ALGORITHM

**For the values given**

- Throughput =  $8000 / 4 = 2000$  bps
- Packet Loss =  $((200 - 180) / 200) \times 100 = 10\%$
- Throughput > 1500 ☒
- Packet loss < 10% ☐

2. An organization has multiple network links connecting its branches.  
The administrator wants to monitor bandwidth utilization and identify overloaded

links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

### **ALGORITHM Bandwidth Monitor**

INPUT n = number of network links

DECLARE usage[n]

WHILE TRUE DO

    FOR i = 1 TO n

    DO

        usage[i] = GET\_BANDWIDTH\_USAGE(i)

    DISPLAY "Link ", i, " Usage = ", usage[i], "%"

    IF usage[i] > 80 THEN

        DISPLAY "Link ", i, " is overloaded!"

        DISPLAY "Recommend switching traffic to backup link."

    ELSE

        DISPLAY "Link ", i, " is operating normally."

    END IF

    END FOR

    WAIT monitoring interval

END WHILE

END ALGORITHM

### **Explanation**

- The algorithm continuously checks the bandwidth utilization of every network link.
- The usage values are stored in an array.
- If utilization exceeds 80%, the link is considered overloaded.
- It recommends switching traffic to a backup link to reduce the load.

- The loop repeats after a fixed monitoring interval.

**How it improves bandwidth management:**

This algorithm helps the administrator identify overloaded links early, distribute traffic to backup links, and prevent network congestion. Continuous monitoring also helps maintain better network performance and efficient utilization of available bandwidth.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

**ALGORITHM Dynamic Network Routing**

```

INPUT n = 6           // Branch offices
INPUT links           // Network links

DECLARE bandwidth [], delay [], utilization []
DECLARE score [], path []

WHILE TRUE DO

    // Collect current network values
    FOR each link i IN links DO
        bandwidth[i] = GET_BANDWIDTH(i)
        delay[i] = GET_DELAY(i)

```

```
    utilization[i] = GET_UTILIZATION(i)
END FOR
```

```
// Calculate quality score
FOR each link i IN links
DO
```

```
    IF bandwidth[i] <= 0 THEN
        score[i] = 0
    ELSE
        score[i] = (bandwidth[i] / (delay[i] +
            1)) * (1 - utilization[i] / 100)
    END IF
```

```
    IF utilization[i] > 80 OR LINK_FAILED(i)
        THEN score[i] = 0
        DISPLAY "ALERT: Link ", i, " overloaded or
        failed" END IF
END FOR
```

```
Find best path from Headquarters to each
branch FOR branch = 1 TO 6 DO
    path[branch] = PATH_WITH_HIGHEST_SCORE (branch, score)
```

```
    IF path[branch] == NONE THEN
        DISPLAY "ALERT: No available path to Branch ", branch
    ELSE
        DISPLAY "Best path to Branch ", branch, " = ",
        path[branch] END IF
END FOR
```

```
WAIT monitoring interval
```

```
END WHILE
```

```
END ALGORITHM
```

**Link Quality Score:**

The algorithm considers bandwidth, delay, and utilization:  $\text{Score} = (\text{Bandwidth} / (\text{Delay} + 1)) \times (1 - \text{Utilization} / 100)$

- **Higher bandwidth** → better score
- **Lower delay** → better score
- **Lower utilization** → better score
- **Failed/overloaded link** → score = 0

**How it improves the network**

1. **Reliability:** Continuously monitors links and avoids failed links.
2. **Routing efficiency:** Selects paths based on overall link quality, not just distance.
3. **Fault tolerance:** Automatically selects an alternate path when a link fails.
4. **Congestion prevention:** Links above 80% utilization are avoided.
5. **Dynamic adaptation:** Recalculates scores periodically, so routing changes as bandwidth, delay, and utilization change.
6. **Administrator alerts:** Reports overloaded or failed links immediately.

**RUBRICS FOR CALCULATION**

| Criteria                  | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|---------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

